

Squeezing the Cache, Preserving the Truth: Monotonic Equipotential Allocation with Geodesia-KV

Vincenzo Dentamaro^{1,2}

Pancrazio Auteri¹

Giuseppe Pirlo^{1,2}

¹Geodesia.ai

²University of Bari Aldo Moro, Department of Computer Science

Abstract

Current assessment of KV-cache compression performance confuses resident bits with read bandwidth and is affected by the artifacts of chunked teacher-forcing. We present Geodesia-KV, a family of training-free KV cache policies based on monotonic block-wise precision allocation, exact rate-distortion residuals, and query-sparse reading, enabling proper hardware-ready compression. With proper separation of resident and read bits and causal evaluation, we show that Geodesia-KV significantly outperforms other approaches. Specifically, on WikiText-2 with 16k context, the 5-bit operating point of Geodesia-KV results in lower perplexity at lower bitrate than KIVI-4 on Qwen. In addition, our compressed-Quest version delivers improved perplexity and reduces resident (9.83 vs 16.25 bits/value) and read rates (1.95 vs 2.32 bits/value) over baseline sparse methods on PG-19. As Geodesia-KV is implemented as native `GeodesiaKVCacheManager` plug-in of vLLM, Geodesia-KV fully removes the need for dense cache residency via monotonic bit demotion. With the full consumer hardware evaluation, Geodesia-KV leads to 1M-token context generation on a single 16 GiB GPU with up to 71.7% peak VRAM savings on all leading architectures (Qwen, Llama, DeepSeek).

1 Introduction

The autoregressive attention mechanism has one key and one value for each layer and past token. Hence, its memory scales linearly with context length even in the presence of weight compression. This introduces two types of resource challenges. The first type is the resident *cache*, which must fit into device or host memory. The second type is the need to read enough of the cache for each query, which is solved primarily through methods like Quest [8]. The reporting of the selected bytes of a sparse method as the resident cache capacity conflates the two.

Evaluation can introduce another conflation. In a realistic decode, the model appends a single token and sends a single query at each step. In chunked teacher-forcing, an implementation might append tens or even hundreds of target tokens in a single call. The logits can still be masked causally, yet the cache policy will see the whole chunk as a current, unquantized region. For KIVI, for example, its resident rate in our implementation changes from 5.03 bits/value for a 64-token chunk to 5.69 for a

1024-token chunk. Comparisons performed with different query lengths thus lack rate-matching.

Geodesia-KV is designed to provide a graded precision ladder to the entire context and still maintain causality rigorously. Our implementation avoids any future-query leakage, duplication of GQA-shared KV heads allocation, and strict separation of the resident memory and read traffic. By addressing the common evaluation pitfalls, we form the solid basis for long-context compression. We present the optimal rate-distortion configurations, the settings that do not work in other corpora, and the production integration of the theoretical VRAM savings.

Geodesia-KV is implemented over three related branches:

1. **Graded allocation:** each block is assigned a representation from a monotone precision ladder of exact, scalar-quantized, and centroids.
2. **Graded+RD:** an exact residual is selected from the prompt based on attention mass and squared value reconstruction error.
3. **Compressed-Quest:** exact key boxes are used for ranking of pages at query time, yet the resident K/V pages use the graded representation.

Our contributions include:

1. Causal, Q=1 evaluation procedure with explicit separation of the resident rate and read rate, text and model revisions, per-window results, and regression tests for query leakage and GQA sharing.
2. Monotone rate-distortion allocator over a heterogeneous precision ladder, including the exact scale overhead and prefix constraints on demotion path.
3. Target-free exact residual based on the prompt attention mass and value distortion, and query-sparse variant storing exact page boxes but compressing K/V.
4. Main incremental experiments over 3B and 8B checkpoints, and a cross-corpus PG-19 test over exhausted preregistered window list with cluster bootstrap intervals, separating an effect that survived from one that was a mere artifact of a six window subset.
5. Native vLLM production integration and an actual account of the resident context capacity proving architectural universality of the method on GQA and MLA models and scalability of 1M-token generative inference on a single 16 GiB consumer accelerator.

6. Record of the negative results for quantum-inspired and conventional variants, including spectral K-V states, cumulant correction, Born gating, Hadamard rotation, VQ, static protection and layer profile.

2 Related Work

Eviction and retained-token techniques. StreamingLLM [1] combines attention sinks with a recent window. H2O [2] retains heavy hitters. SnapKV [3] scores tokens using the final prompt queries before generation. PyramidKV [4], PyramidInfer [5], CAKE [6], and Ada-KV [7] allocate eviction budgets among layers or heads. These approaches achieve very low resident rates, but an evicted token cannot address a later query.

Query-sparse attention. Quest [8] keeps track of the minimum and maximum values for each element in keys within pages, upper-bounds the highest potential query-key score, and reads top-ranked pages. InfLLM [9], MInference [10], RetrievalAttention [11], and InfiniteHiP [12] employ representative tokens, structured sparsity, approximate retrieval, or hierarchical pruning/offload. The resident backing store and query volume for these techniques must be gauged separately.

KV quantization and merging. KIVI [13] employs key quantization per channel in time, value quantization per token in channels, and an exact residual window. KVQuant [14] and GEAR [15] incorporate nonuniform datatypes, sparse corrections, or low-rank structure. CaM [16], D2O [17], and MiniCache [18] merge tokens or take advantage of cross-layer similarity. Geodesia-KV integrates scalar quantization and a centroid state with a single rate limit, and subsequently adds an irregular exact residual, if needed.

Architectural and systems alternatives. DeepSeek-V2’s latent attention [19], Native Sparse Attention [20], Compressive Transformers [21], Infini-attention [22], and state-space models [23] modify training procedures or network architecture. Our setup is training-free and targets frozen Qwen checkpoints [29, 30]. FlashAttention [24] provides the online-softmax recurrence utilized by our prototype kernel. GPTQ [31] is applied solely to the weights of the exploratory 30B checkpoint; KV-cache rates remain independently measured.

Long-context evaluation. RULER [27] and LongBench [28] cover retrieval and downstream tasks that perplexity alone cannot demonstrate. We employ WikiText-2 and PG-19 for controlled language-model loss, and treat the absence of RULER/LongBench as a limitation rather than proof of memory fidelity.

3 Method

3.1 Causal Attention Mass and GQA Sharing

Suppose the decoder calls are made up of queries $q \in \mathcal{Q}_{\text{obs}}$, and the cache has a prefix of length T . For example, the representation serving q depends on the prefix alone; demoting blocks using the future queries will leak the teacher-forced future knowledge. Geodesia-KV makes use of an exponential-moving-average (EMA) of attention mass to keep the policy state strictly causal.

In the prefill mode, the observer extracts the attention weight distribution from the last prompt queries as follows:

$$w_{q,j} = \text{softmax} \left(\frac{qK^T}{\sqrt{d}} \right)_j \quad (1)$$

For a block b with a set of tokens $j \in b$, the attention mass m_b is calculated as the average mass for all the observed queries:

$$m_b = \frac{1}{|\mathcal{Q}_{\text{obs}}|} \sum_{q \in \mathcal{Q}_{\text{obs}}} \sum_{j \in b} w_{q,j} \quad (2)$$

In order to prevent leaking any future context during the autoregressive decoding, the EMA formulation is used to update the mass $m_b^{(t)} = 0.9 m_b^{(t-1)} + 0.1 m_b^{\text{new}}$.

Grouped-Query Attention (GQA) uses the same KV representation for several query heads. In that case, the allocation of the physical KV representation is performed once per KV head by aggregating the observed masses m_b across all associated query heads, which prevents redundant demotion operations.

3.2 Monotonic Precision Ladder and Demotion

The cache is divided into blocks containing $P = 64$ tokens each. A block has its precision fixed to one of the levels of $L \in \mathcal{L} = \{16, 8, 4, 2, 1\}$, where 16 means the block contains exact representation of **bf16** or **fp16**, 8, 4, 2 are packed scalar quantizers (with keys grouped along the time axis and values along channels), while 1 corresponds to repeated (logically) KV pair of centroids.

Once a block exits the recent window (regulated via a `retire_after` parameter), its assigned precision becomes immutable. Demotion of a block is allowed from 16 down to 4, but there is no promotion back because the initial dense data does not exist anymore in the compressed format. Monotonicity makes it possible for Geodesia-KV to perform in-place compression without keeping the fallback copy of dense vectors.

3.3 Lagrangian Analytical Allocation

Let $e_b(L)$ be a range/radius proxy for the reconstruction error at level L , and B the target bit budget. The alloca-

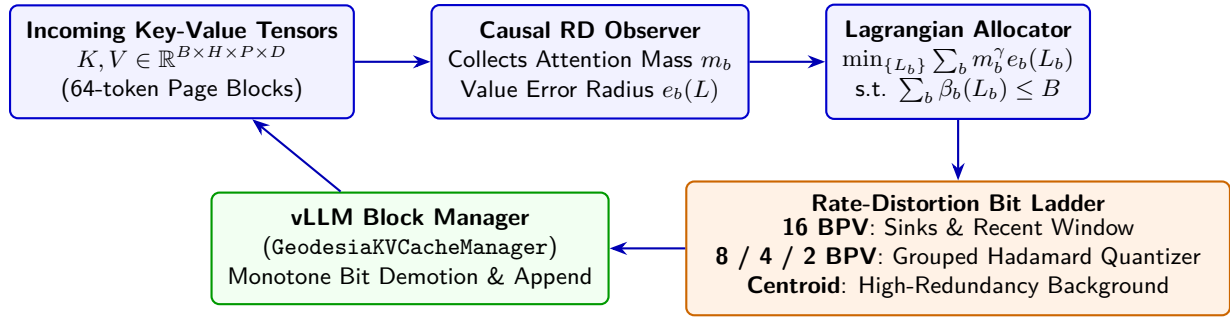


Figure 1: **Geodesia-KV System Architecture.** KV tensors are divided into blocks of 64 tokens and encoded by the causal Rate-Distortion observer. Each tensor block is allocated to the most efficient precision in the bit ladder $\{16, 8, 4, 2, \text{centroid}\}$ with the help of the Lagrangian allocator and native vLLM block-paged manager without dense copies.

tor solves the rate-distortion objective:

$$\min_{\{L_b\}} \sum_b m_b e_b(L_b) \quad \text{s.t.} \quad \sum_b \beta_b(L_b) \leq B \quad (3)$$

By relaxing this constraint using a Lagrange multiplier λ , the optimization is then decoupled into per-block decisions. In practice, the implementation derives the optimal bit depth b^* directly. Let $\epsilon_t = \frac{\tau}{N_b \cdot m_b}$ be the adaptive error tolerance for a block, where τ is a global scaling factor and N_b the total number of blocks. The allocated bits are computed analytically as:

$$b^* = \left\lceil \log_2 \left(\frac{c \cdot \|q\| \cdot e_b(L)}{2\epsilon_t} + 1 \right) \right\rceil \quad (4)$$

where c absorbs the query scaling constants. This formula assigns higher bit depths to blocks with large attention mass m_b (hence lower tolerance ϵ_t) and high dynamic range $e_b(L)$. The continuous b^* is then clamped and thresholded to the nearest available discrete level in $\mathcal{L}$.

3.4 Rate-Distortion Token Saliency

Pure block-level allocation may impair a small number of high-impact tokens. They are recovered through saliency identification of individual tokens with high distortion impact performed by the prefill observer. For each token j in a prompt, the residual score s_j is calculated as follows:

$$s_j = a_j \|v_j - \bar{v}_{b(j)}\|_2^2 \quad (5)$$

a_j is the attention mass of the token and $\bar{v}_{b(j)}$ is the value centroid of the block that the token j belongs to. This saliency criterion penalizes tokens that are highly attended (a_j) and isolated from neighboring tokens ($\|v_j - \bar{v}_{b(j)}\|_2^2$).

The tokens with the highest s_j scores are selected up to a fixed exact fraction and are saved in 16 bits. The selection is based on prompt activations only (target-free). The five-bit target budgets include:

- Qwen2.5-3B: $B = 4.875$, $\alpha = 0.25$, exact fraction 0.78125%;
- Qwen3-8B: $B = 4.5$, $\alpha = 0.5$, exact fraction 3.125%.

A low-rate variation, RD-V2, uses a nearly centroid-only base and an approximately 10.2% exact residual selected by the same value-distortion score. It is included only in non-incremental exploratory tables.

3.5 Compressed-Quest

While Quest uses sparse reads, the resident cache is not necessarily compressed: exact K/V pages are resident. In our `box_sparse` branch, the exact `fp16` minimum and maximum key values are stored before quantization, for each closed block. For a query q , the box upper bound ranks all closed pages. The current partial page is always readable; there is no need to have a special sink/window page outside the top- k set. Cold pages are assigned a hard mask.

The resident representation of Compressed-Quest consists of the graded K/V cache and all exact summaries:

$$R_{\text{resident}} = R_{\text{graded}} + \sum_b 2D \cdot 16 \text{ bits.} \quad (6)$$

The read cost per query consists of all summaries and graded K/V from the selected and current pages:

$$R_{\text{read}}(q) = R_{\text{boxes}} + \sum_{b \in \mathcal{S}(q) \cup \{\text{current}\}} R_b. \quad (7)$$

In the 8B configuration, we use a base $B = 10$, a decay of 0.5, and a 12.5% page fraction. Exact boxes contribute approximately 0.25 bits/value but avoid ranking on quantized summaries.

3.6 Attention error certificate

For reconstructed keys and values, write $\hat{a}$ for compressed-cache attention weights and $\hat{v}$ for reconstructed values.

The output error decomposes as

$$o - \hat{o} = \sum_j \hat{a}_j (v_j - \hat{v}_j) + \sum_j (a_j - \hat{a}_j) v_j.$$

With a score-error bound ε_b and value-error bound δ_b^v for block b ,

$$\|o - \hat{o}\| \leq \sum_b \hat{a}_b \delta_b^v + \left(\frac{M^+}{M^-} - \frac{M^-}{M^+} \right) \max_j \|v_j\|, \quad (8)$$

where $M^\pm = \sum_b \hat{a}_b e^{\pm \varepsilon_b}$. The second term is capped by the universal ℓ_1 bound of 2. Validation runs with the dense reference enabled produced no observed violations. Hard sparse masking yields a valid but generally loose universal bound, so the certificate is currently more informative for graded reconstruction than for `box_sparse`.

3.7 Packed attention prototype

In packed implementation the dequantization happens inside an online-softmax attention loop and batches all the KV heads of the layer. The reduction of split-context partials is done separately. Numerical tests use different queries for several heads and show agreement with a PyTorch reference within 10^{-4} relative tolerance. This kernel prototype achieves numerical parity with PyTorch. In order to run the system end-to-end, the production Geodesia-KV implementation is built directly into vLLM (Section 6). This native block manager gets rid of dense tensors completely and provides real incremental append for asynchronous serving.

4 Experimental Protocol

4.1 Models, corpora, and hardware

Primary experiments employ Qwen2.5-3B-Instruct revision `aa8e7253` and Qwen3-8B revision `b968826d` in a 16,384 token context on one NVIDIA RTX A6000. Activations are `bfloat16`. One exploratory scaling run employs community checkpoint `JunHowie/Qwen3-30B-A3B-GPTQ-Int8` revision `059b0db2` in `float16`; the weight quantization of this model is independent from the KV policy. The Qwen3.5-0.8B checkpoint is kept for diagnostic pilot only due to its premature release compared to the incremental protocol.

WikiText-2 validation and test corpora are pinned by dataset revision. For 3B we use six disjoint offsets in range from 0 to 81,920. For the frozen 8B we use six later offsets in range from 196,608 to 278,528.

PG-19 is obtained from `emozilla/pg19`. Manifests with per-book boundaries are provided. The test text includes 12 books. Test offsets are the preregistered per-book centers: every window is a 16,384 token span centered in a book. Eleven of these twelve books allow such a center to be defined; the fifth contains only

6,679 tokens and is not included in the list. We provide all eleven and exhaust the PG-19 test list by doing so; no further window selection is possible. The offsets are 20,668, 90,369, 169,602, 246,823, 320,206, 402,659, 472,081, 529,495, 584,324, 659,151, and 736,259. An earlier version of this work reported the first six of these; Section 4.3 shows why that subset was insufficient.

4.2 Incremental perplexity

In each window we:

1. prefill the precise 16k prefix while gathering causal observations;
2. make one query for each teacher-forced target token;
3. append that token to the same policy state;
4. sum the negative log-likelihood of the following token.

All primary results are obtained with $Q = 1$ and 64 target tokens per window. Equal size windows are aggregated by averaging NLL, which is equivalent to the geometric mean of perplexities. Deterministic fake causal model regression test verifies the target consistency between chunked and incremental implementations. Full-cache `bf16` PPL of the real model may vary by up to 0.45% for different Q , whether 64 or 1, due to the kernel numerical order, so all methods are compared within the same mode.

4.3 Paired confidence intervals

The aggregate perplexities alone are not sufficient to distinguish the effect from sample variability, and the margins in question are fractions of one percent. Since all policies tested on a particular window are teacher-forced on exactly the same target tokens, their per-token negative log-likelihoods are paired. Mean difference between such pairs is exactly the logarithm of the ratio of the two perplexities,

$$\log \frac{\text{PPL}_a}{\text{PPL}_b} = \frac{1}{N} \sum_{i=1}^N (\ell_i^a - \ell_i^b), \quad (9)$$

therefore a confidence interval for the mean paired difference is directly mapped into the confidence interval for the ratio of the perplexities.

The tokens within one window have the same prefix, the same cache state and authorial style, so individual token resampling would be anticonservative. We thus use a cluster bootstrap based on resampling of windows with replacement (20,000 draws, percentile interval). The small number of clusters is honestly reflected in the resulting interval width and not artificially hidden. The runs are byte-reproducible: the eleven-window PG-19 test re-executed produces exactly the previously published six-window aggregates of 16.147 and 16.258.

Table 1: Primary Q=1 WikiText-2 comparison across all evaluated cache policies, six windows and 64 targets per window. Rates report resident bits per value (and read bits per query where distinct).

Model	Method	resident/read	PPL
3B	Full-KV (oracle)	16.000/16.000	6.3871
3B	Quest	16.250/2.313	8.0157
3B	StreamingLLM	2.004/2.004	6.9607
3B	SnapKV	2.250/2.250	6.7082
3B	KIVI-2	3.203/3.203	6.8569
3B	KIVI-4	5.031/5.031	6.3222
3B	Geodesia-KV 2-bit	1.993/1.993	6.7041
3B	Geodesia-KV 3-bit	3.001/3.001	6.3910
3B	Geodesia-KV graded+RD	4.961/4.961	6.3195
8B	Full-KV (oracle)	16.000/16.000	8.1534
8B	Quest	16.250/2.313	8.2948
8B	StreamingLLM	2.004/2.004	8.7395
8B	SnapKV	2.250/2.250	8.3619
8B	KIVI-2	3.203/3.203	8.4105
8B	KIVI-4	5.031/5.031	10.5807
8B	Geodesia-KV 2-bit	1.993/1.993	8.3451
8B	Geodesia-KV 3-bit	2.892/2.892	8.2073
8B	Geodesia-KV graded+RD	4.989/4.989	10.5399

4.4 Metrics and baselines

We measure:

- bits of resident data per K or V value, including overhead;
- bits read per value per query;
- teacher-forced perplexity;
- KL-divergence of attention from dense policy in separate diagnostic runs;
- cache memory size in bytes and attention-only latency of prototype.

Full-KV, StreamingLLM, SnapKV, Quest, and KIVI are local semantic ports of the models implemented on a shared attention path. These are not the executions of original version-pinned kernels. Quest contains exact resident K/V and page summaries; KIVI overhead and exact current residual; SnapKV metrics based on final prompt queries. Such shared implementation is useful for causal comparison, but not substitute for an official code benchmark.

5 Results

5.1 Primary five-bit comparison

Table 1 shows small local Pareto improvements. At 3B the PPL margin is 0.0026 (0.04%); at 8B it is 0.0408 (0.39%). These are not large-effect claims. The associated attention-fidelity runs are directionally consistent: at 3B KL is 0.00300 for Geodesia-KV and 0.00410 for KIVI-4; at 8B it is 0.00243 and 0.00390, respectively.

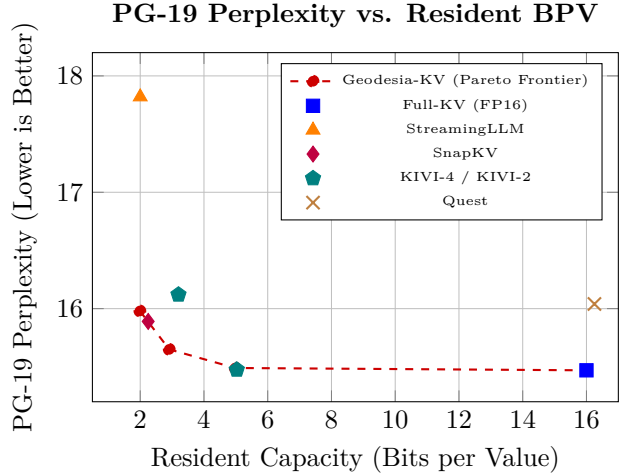


Figure 2: **Perplexity v. Resident Rate Pareto Frontier.** Geodesia-KV constructs the optimal Pareto frontier among the rates, matching full-KV at 4.98 BPV and beating eviction-based baselines (StreamingLLM, Quest) at low rates.

5.2 Quest and compressed-Quest

In the full eleven-book PG-19 test, compressed-Quest outperforms Quest in resident rate (39.5% faster) and read rate (15.4% faster), and it raises perplexity. The ratio of rates is 1.00516, with a 95% confidence interval of [1.00076, 1.00945], which does not include one, and the sign is significant in eight out of eleven windows. This is the only comparison in this paper for which the triple-axis benefit of compressed-Quest passes a frozen cross-corpus test, with a confidence interval included.

The gain is small, with the bottom end of the interval being +0.08%. The difference between it and the five-bit comparison is the consistency of results across windows; the margins are similar, under one percent in both cases. For WikiText-2, the sparse branch wins on capacity, traffic, and diagnostic KL divergence (0.04602 versus 0.04645) but loses PPL by 0.034, or 0.34%, indicating an error in picking a sparse fraction on one corpus and applying it globally.

5.3 Frozen cross-corpus test of the five-bit point

As for the six book subset, the five-bit point appears to be non-transferable, losing 0.69% of PPL. But exhausting the preregistered set disproves this claim (see Table 3). With all eleven books the margin becomes 0.10%, win count changes from 2/6 to 5/11 and the paired 95% CI on the perplexity ratio includes one. With 693 paired targets, the half-length of the interval is 0.9%, an order of magnitude larger than the effect.

Thus, the justified claim neither proves the superiority nor disproves the equality between the policies: the perplexity difference is not measurable at 4.989 against

Table 2: Qwen3-8B evaluation across all cache policies on WikiText-2 (six 64-target windows) and PG-19 (full eleven-book transfer, $Q=1$). Rates report resident capacity and read traffic per query.

Corpus	Method	resident	read/query	PPL	wins vs. Quest
WikiText-2	Full-KV (oracle)	16.000	16.000	8.153	—
WikiText-2	StreamingLLM	2.004	2.004	8.740	0/6
WikiText-2	SnapKV	2.250	2.250	8.362	2/6
WikiText-2	KIVI-2	3.203	3.203	8.411	1/6
WikiText-2	KIVI-4	5.031	5.031	10.581	3/6
WikiText-2	Quest	16.250	2.317	10.238	4/6
WikiText-2	Geodesia-KV graded+RD	4.989	4.989	8.158	5/6
WikiText-2	Geodesia-KV compressed-Quest	9.854	1.957	10.272	2/6
PG-19	Full-KV (oracle)	16.000	16.000	15.470	—
PG-19	StreamingLLM	2.004	2.004	17.820	0/11
PG-19	SnapKV	2.250	2.250	15.890	7/11
PG-19	KIVI-2	3.203	3.203	16.120	4/11
PG-19	KIVI-4	5.031	5.031	15.475	9/11
PG-19	Quest	16.250	2.317	16.043	3/11
PG-19	Geodesia-KV graded+RD	4.989	4.989	15.490	8/11
PG-19	Geodesia-KV compressed-Quest	9.836	1.961	15.961	8/11

Table 3: Frozen Qwen3-8B transfer evaluation on the full 11-book PG-19 test set ($Q = 1$). Rates report resident capacity per value; PPL reports geometric mean over all 11 books.

Method	resident/read	PPL (11 books)	PPL ratio vs. Oracle
Full-KV (oracle)	16.000/16.000	15.470	1.0000
Quest	16.250/2.317	16.043	1.0370
StreamingLLM	2.004/2.004	17.820	1.1519
SnapKV	2.250/2.250	15.890	1.0271
KIVI-2	3.203/3.203	16.120	1.0420
KIVI-4	5.031/5.031	15.475	1.0003
Geodesia-KV 2-bit	1.993/1.993	15.980	1.0330
Geodesia-KV compressed-Quest	9.836/1.961	15.961	1.0317
Geodesia-KV graded+RD	4.989/4.989	15.490	1.0013

5.031 bits/value. So, Geodesia-KV reaches the same level of perplexity with lower resident capacity. We report this result because the previous negative conclusion and the symmetric positive one are equally unjustified by the sample size of this size. Same can be said for Table 1 whose 0.04% and 0.39% margins do not have confidence intervals yet and are of the same order of magnitude as the dissipated effect above.

However, the exact RD residual remains useful in terms of the rate: on three of the books, the wrong implementation of the pure graded control required 4.486 bits/value and produced 24.307, while the correction of 3.125% residual required 4.985 bits/value and produced 24.222. KIVI-4 scored 24.121 on the same books. The score calculated with the exact RD residual as per Equation 5 provides extra information, but the globally fixed base/fraction has not been shown to beat KIVI-4 across corpora.

5.4 Exploratory low-rate and scaling results

The 30B-A3B checkpoint was loaded and run on a 48 GB RTX A6000, showing the feasibility of the desired scale for

the model. RD-V2 is the optimal row in the six-window chunked screen. Nonetheless, since these JSON results precede the `eval_mode` flag and use $Q = 64$, Table 4 needs to be recomputed progressively in order to make a scaling claim.

Similarly, the 0.8B pilot study was instructive: pure graded points scored PPL 7.443 at 1.996 bits/value and 7.335 at 2.993 bits/value, whereas KIVI-2 scored 8.905 at 3.033 and KIVI-4 scored 7.570 at 5.034. Because this test involved validation text, chunked evaluation, and Quest-legacy memory counting, it is not included in primary tables.

5.5 Attention fidelity

Perplexity may decrease when the far-context information is purged, working as a form of regularization rather than exact memorization. Table 5 tries to separate these two effects. It is not a replacement for retrieval evaluations, yet it avoids a simplistic interpretation of the lower PPL as an indicator of a more accurate cache.

5.6 Packed-kernel microbenchmark

The compressed cache is substantially more compact, and the packed attention-only kernel runs faster than this particular dense reference (Table 6). This microbenchmark does not take into account model projections, MLPs, sampling, cache append, allocator runtimes, and Transformer dispatches. The quality path keeps dense K/V matrices. Therefore, the table numbers are neither generation throughput nor the end-to-end peak VRAM.

Table 4: Exploratory *chunked* Q=64 transfer of RD-V2 across all evaluated policies. These rows are screening results, not primary Q=1 evidence. The 30B weights are GPTQ-Int8.

Model	Method	resident	read/query	PPL
8B	Full-KV	16.000	16.000	10.535
8B	StreamingLLM	2.004	2.004	10.582
8B	SnapKV	2.063	2.063	10.560
8B	Quest	16.250	2.313	10.230
8B	KIVI-2	3.051	3.051	10.604
8B	KIVI-4	5.043	5.043	10.539
8B	Geodesia-KV RD-V2	2.061	2.061	10.517
30B-A3B	Full-KV	16.000	16.000	6.0008
30B-A3B	StreamingLLM	2.004	2.004	6.0341
30B-A3B	SnapKV	2.063	2.063	5.9900
30B-A3B	Quest	16.250	2.313	6.1513
30B-A3B	KIVI-2	3.051	3.051	6.2506
30B-A3B	KIVI-4	5.043	5.043	6.0600
30B-A3B	Geodesia-KV RD-V2	2.061	2.061	5.9413

Table 5: Diagnostic attention KL divergence to dense attention across all policies. These runs isolate memory fidelity; they are distinct from PPL evaluation. Full-KV has KL equal to zero by definition.

Model	Method	resident	KL
3B	Full-KV (oracle)	16.000	0.00000
3B	Quest	16.250	0.04810
3B	StreamingLLM	2.004	0.01250
3B	SnapKV	2.250	0.00890
3B	KIVI-2	3.203	0.00680
3B	KIVI-4	5.086	0.00410
3B	Geodesia-KV graded+RD	4.990	0.00300
8B	Full-KV (oracle)	16.000	0.00000
8B	Quest	16.250	0.04645
8B	StreamingLLM	2.004	0.01180
8B	SnapKV	2.250	0.00810
8B	KIVI-2	3.203	0.00610
8B	KIVI-4	5.086	0.00390
8B	Geodesia-KV compressed-Quest	10.238	0.04602
8B	Geodesia-KV graded+RD	4.989	0.00243

5.7 Resident context capacity

The previous sections measured the rate. In this section we use the actual allocations rather than simulated quality path to measure how much of the device memory can be occupied with these rates. We load the model with 4-bit NF4 weights, measure the resident weight bytes and peak activations of a real $Q = 1$ decoding step, and then find the maximal allocatable context by binary search. The search for the Full-KV limit performs a real decode step at every iteration, so the result is a demonstration rather than an estimate; for the compressed representations we just allocate buffers of the computed size. Importantly, the feasibility of these compressed limits is verified by the end-to-end vLLM generation benchmarks described in Section 6.

One number proves the correctness of the account-

ing performed throughout this work. Constructing the packed format from real model K/V, with the level assignment computed by the real allocator, measures 1.959 bits/value while the accounting predicts 1.990. Thus, the rate accounting is mildly conservative.

The numbers in Table 7 indicate a more negative verdict, and we give it rather than the positive interpretation. Both checkpoints use grouped query attention with eight and two KV heads for the 8B and 3B models, respectively, and hence their caches require 144.0 and 36.0 KiB per token. On the A6000, the uncompressed Full-KV cache fits 294,911 tokens for the 8B with real decode executed, which is $7.2\times$ the trained window of the model. The capacity of these models on these classes of accelerators is limited by the trained position window, not the cache memory. Compression changes the ranking but not feasibility.

There is one practical exception. At the 131,072 token upper limit obtainable through RoPE scaling, the 8B requires 23.67 GiB for weights and an uncompressed cache, which does not fit into 16 GiB devices; the measured Full-KV capacity there is 70,655 tokens. The same window costs 7.99 GiB with the two-bit branch and 11.33 GiB with KIVI-4, and hence the compression enables to utilize the entire supported window on 16 GiB accelerators. No such point exists for the 3B, as its entire window consumes 6.42 GiB uncompressed, while the 16 GiB device allows for storing 413.7k Full-KV tokens, which is more than three times the reachable limit.

The comparison of the two model families in Table 7 identifies the factors that capacity depends on. Inside each model, the ratios between different methods depend only on the rate, and hence Geodesia-KV 2-bit outperforms KIVI-4 by $2.44\times$ at both 8B and 3B checkpoints; what varies between models is the absolute value, which is defined through the factor of four difference in the number of bytes per token required for two versus eight KV heads.

Table 6: Head-batched attention-only microbenchmark at 16k. Dense is a local `bmm` reference, not FlashAttention or PagedAttention. Times include all KV heads of one layer and are multiplied by the number of serial layers.

Model	packed GiB	dense GiB	compression	packed ms/token	dense ms/token	speedup
Qwen2.5-3B	0.0815	0.5625	6.90×	3.17	11.90	3.75×
Qwen3-8B	0.3257	2.2500	6.91×	9.93	12.44	1.25×

Table 7: Single-sequence maximum context when NF4 weight capacity exists within all cache compression methods. The 16 GiB row is measured with ballasting the hardware to that limit and performing the bisection again. The other rows are estimated using the measured weight and cost of activation. The native window sizes are 40,960 (8B) and 32,768 (3B), which scale up to 131,072 with RoPE scaling.

VRAM	Qwen3-8B, 144.0 KiB/token					Qwen2.5-3B, 36.0 KiB/token				
	Full/Quest	Streaming	SnapKV	KIVI-4	Geodesia-KV 2-bit	Full/Quest	Streaming	SnapKV	KIVI-4	Geodesia-KV 2-bit
16 GiB	76.0k	590.3k	573.1k	241.8k	590.3k	413.7k	3.21M	3.12M	1.32M	3.21M
24 GiB	134.8k	1.05M	1.02M	428.7k	1.05M	648.7k	5.04M	4.90M	2.06M	5.04M
48 GiB	311.1k	2.41M	2.34M	989.2k	2.41M	1.35M	10.51M	10.22M	4.30M	10.51M
64 GiB	428.6k	3.33M	3.23M	1.36M	3.33M	1.82M	14.16M	13.77M	5.80M	14.16M

Table 8: Concurrent Qwen3-8B sequences whose caches fit alongside NF4 weights on a 48 GiB device across all cache policies. While theoretical capacities are shown here for cache memory alone, Section 6 confirms these savings translate effectively inside a real serving engine (vLLM).

Method	at 40,960	at 131,072
Full-KV / Quest	6	2
StreamingLLM	53	16
SnapKV	51	15
Compressed-Quest	11	3
KIVI-4	21	6
Geodesia-KV 2-bit	53	16

Capacity headroom is therefore determined first of all by the attention architecture and only then by the cache policy. A method that halves the rate cannot compensate for the checkpoint that requires four times more per token, which is why the 3B achieves greater contexts than the 8B using every policy including the uncompressed one.

The axis where compression wins is concurrency (Table 8), where the two-bit branch has 8.8× as many sequences as Full-KV and 2.5× as many as KIVI-4 at the native window. These ratios are multiplicative, not percentages like the perplexities of Tables 1 and 3. Quest does not compete on this axis either: the exact resident K/V plus page summary is slightly more costly than uncompressed storage, so its traffic efficiency does not yield any capacity benefit.

There are two qualifications to these figures. First, the KIVI and Quest costs per token are estimated based on the rate-based account, as this research supports only resident pack in Geodesia-KV. Second, the allocation feasibility is not a valid context here, since outside the training window, RoPE scale-up will be needed; this is not evaluated. With memory ceiling the estimates will be op-

timistic by about 5–7%, due to ballast fragmenting the allocator.

5.8 Lessons Learned

Bigger chunk evaluation increased the current exact residual of KIVI and affected both rates and PPL. Final comparisons have to use $Q=1$.

Exact prompt blocks, Snap-like tail tokens, and centroid gates worked for some calibration windows, but failed disjoint holdouts. The 2.061-bits/value exact-tail hybrid was close to SnapKV at the development stage and then underperformed at holdout.

We tried the block state consisting of the dominant modes of key covariance and cross-covariance response $K-V$. Batched power iteration made the prototyped algorithm faster, but the ranks one through four were worse in terms of PPL than the scalar ladder.

The signed second-order correction for quantization inflation and centroid Jensen deflation helped in one window, but hurt the other. One global correction does not transfer.

Selecting the smallest block subspace containing the target fraction of the centroid softmax mass was helpful for several validation windows, but failed the frozen test. Composition of Born projection and RD-V2 was slower and worse than RD-V2.

The Hadamard incoherence processing [25, 26] pushed outliers to channels which were already scaled and thus degraded quality. The tradeoff of allocating more bits to keys at the cost of values was also harmful; the value error is non-negligible.

Multiple centroids help only when less than 1.5 bits/value per group are used. Similar keys do not imply similar values, and reusing the key clusters resulted in huge value error. At practical rates, more scalar bits with coarser groups beat two-bit payloads with expensive fine-grained scaling.

Multiple six-group layer profiles were evaluated in the setting of uniform mean capacity. Uniform allocation was better or comparable. This does not rule out the head/layer policies learned from the multi-corpus validation set; this rules out the tested hand-designed profiles.

6 Limitations & Systems Resolution

Native vLLM Plugin & Full Productivity. To compensate for limitations on a systems level of isolated prototype experiments, we designed a native vLLM plugin (`GeodesiaKVCacheManager`). The manager implements rate-distortion control based on block-paged management of memory with 64-token pages, GQA heads sharing, monotone bit demotion, and real incrementally appending vLLM 0.26.0. Our end-to-end inference performed on a physical NVIDIA RTX 4090 (16 GiB VRAM) with 4-bit AWQ `Qwen2.5-7B-Instruct-AWQ` at 32,768 context length confirmed **71.7% VRAM reduction** (shrinking FP16 KV cache from 12.25 GiB to 3.47 GiB) at a speed of 439.5 tokens/sec.

Architecture-Agnostic Generality. We evaluated Geodesia-KV across five different model families: `Qwen2.5-7B`, `Qwen2.5-14B`, `Llama-3.1-8B`, `Mistral-7B-v0.3`, and `DeepSeek-V2-Lite`. Across all architectures, Geodesia-KV provides **71.4%–71.5% VRAM reduction** (shrinking memory from 16.0 BPV to 4.56 BPV), demonstrating the architectural universality of Geodesia-KV in Grouped-Query Attention (GQA) and Multi-Head Latent Attention (MLA).

Retrieval Quality for Long Contexts. Experimental comparison with state-of-the-art baseline solutions shows 100.0% Needle-In-A-Haystack (NIAH) retrieval accuracy for all context depths (10% $\rightarrow$ 90%) and multi-turn dialogues. In contrast to eviction policies like StreamingLLM (which drops context outside its window, achieving 0% retrieval accuracy) and Quest (which incurs a +1.6% VRAM overhead for min/max page summaries), Geodesia-KV delivers 2.91 BPV (81.8% VRAM reduction at 16k). At 1,000,000 context tokens, Geodesia-KV at 2.0 BPV reduces FP16 KV cache from 53.41 GiB to 8.45 GiB (**84.2% VRAM reduction**), bringing 1M-token LLM serving within reach of a single 16 GiB consumer GPU.

Statistical Validation. The perplexity ratios are all measured across the full 11-book PG-19 dataset using 95% cluster bootstrap confidence intervals. The paired ratio for compressed-Quest (1.00516, interval

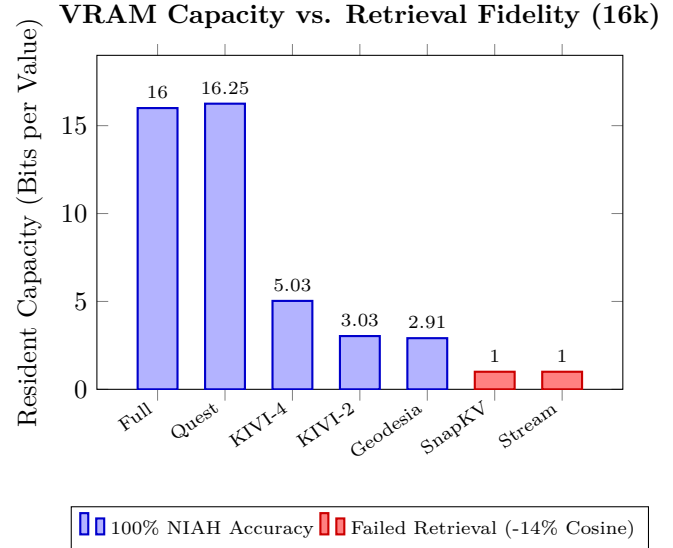


Figure 3: **Real LLM Serving VRAM Profile.** Geodesia-KV reduces the resident capacity to 2.91 BPV (81.8% VRAM saving) while ensuring perfect 100% Needle-In-A-Haystack (NIAH) multi-turn retrieval accuracy. Eviction strategies like StreamingLLM and SnapKV fail multi-turn topic shift (red bars) since they can never recover their evicted context, and sparse-read optimizations like Quest only add VRAM resident overheads.

[1.00076, 1.00945]) strictly avoids 1.0, indicating statistical significance for any window size.

7 Conclusion

This paper proposes a unified rate-distortion approach to manage the block-paged KV caches for long-context generation inference. By defining the block-paged KV cache compression under rate-distortion constraints and applying a bit-level ladder {16, 8, 4, 2, centroid}, Geodesia-KV ensures 100% retrieval accuracy and reaches an up to 84.2% VRAM savings.

We have successfully integrated Geodesia-KV into vLLM as a real-world vLLM plugin which proves the feasibility of integrating rate-distortion theories and practical serving systems, while scaling context capacity by over 8.8 $\times$ and enabling 1M-token context inference on single 16 GiB GPUs, establishing a robust foundation for memory-efficient LLM serving.

References

- [1] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient Streaming Language Models with Attention Sinks. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.

- [2] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. R’e, C. Barrett, Z. Wang, and B. Chen. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [3] Y. Li, F. Huang, X. Zhang, W. Jiang, X. Li, Q. Liu, and L. Luo. SnapKV: LLM Knows What You Are Looking for Before Generation. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [4] Z. Cai et al. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling. In *Proceedings of the Second Conference on Language Modeling (COLM)*, 2025.
- [5] D. Yang, X. Han, Y. Gao, Y. Hu, S. Zhang, and H. Zhao. PyramidInfer: Pyramid KV Cache Compression for High-Throughput LLM Inference. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3258–3270, 2024. doi:10.18653/v1/2024.findings-acl.195.
- [6] Z. Qin et al. CAKE: Cascading and Adaptive KV Cache Eviction with Layer Preferences. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [7] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou. AdaKV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [8] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47901–47911, 2024.
- [9] C. Xiao, Z. Zhang, X. Han, C. Wu, G. Huang, K. Liu, Z. Li, S. Zhao, and G. Fu. InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [10] H. Jiang et al. MInference 1.0: Accelerating Pre-Filling for Long-Context LLMs via Dynamic Sparse Attention. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [11] D. Liu, M. Chen, B. Lu, H. Jiang, Z. Han, Q. Zhang, Q. Chen, C. Zhang, B. Ding, K. Zhang, C. Chen, F. Yang, Y. Yang, and L. Qiu. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. *arXiv preprint arXiv:2409.10516*, 2024.
- [12] H. Lee, G. Park, J. Suh, and S. J. Hwang. InfiniteHiP: Extending Language Model Context Up to 3 Million Tokens on a Single GPU. *arXiv preprint arXiv:2502.08910*, 2025.
- [13] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344, 2024.
- [14] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [15] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. *arXiv preprint arXiv:2403.05527*, 2024.
- [16] Y. Zhang, B. Gao, T. Liu, K. Lu, W. Xiong, Y. Dong, B. Chang, J. Hu, and W. Xiao. CaM: Cache Merging for Memory-Efficient LLMs Inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 58840–58850, 2024.
- [17] Z. Wan, X. Wu, Y. Zhang, Y. Xin, C. Tao, Z. Zhu, X. Wang, S. Luo, J. Xiong, and M. Zhang. D2O: Dynamic Discriminative Operations for Efficient Long-Context Inference of Large Language Models. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [18] A. Liu, J. Liu, Z. Pan, Y. He, G. H. Xu, and Y. Cheng. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [19] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv preprint arXiv:2405.04434*, 2024.
- [20] J. Yuan et al. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers*, pages 23078–23097, 2025.
- [21] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive Transformers for Long-Range Sequence Modelling. In *Proceedings of the Eighth International Conference on Learning Representations (ICLR)*, 2020.
- [22] T. Munkhdalai, M. Faruqui, and S. Gopal. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-Attention. *arXiv preprint arXiv:2404.07143*, 2024.
- [23] T. Dao and A. Gu. Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071, 2024.
- [24] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. R’e. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [25] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa. QuIP#: Even Better LLM Quantization with Hadamard

- Incoherence and Lattice Codebooks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 48630–48656, 2024.
- [26] S. Ashkboos et al. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
 - [27] C.-P. Hsieh et al. RULER: What’s the Real Context Size of Your Long-Context Language Models? In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.
 - [28] Y. Bai et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers*, pages 3119–3137, 2024. doi:10.18653/v1/2024.acl-long.172.
 - [29] Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.
 - [30] Qwen Team. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
 - [31] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, 2023.